

CS361 Computer Architecture

Assignment 1

Covers Trends in cost/performance+ Amdahl's Law + OpenMP (Programming)

Total Marks: 100

Collaboration: Individual (no code sharing).

Due date: 8 Oct 2025 11:59PM

Submission Pack (Single ZIP):

- A1_Report.pdf (answers, tables, and short explanations)
- code/omp_assignment.c (completed starter)
- results/measurements.csv (timings & speedups)
- System info text file (OS, CPU model, number of cores/threads)

Environment & Build (Linux/macOS/WSL recommended):

- Run with different threads: `OMP_NUM_THREADS=4 ./omp_assignment`
- If memory-limited, you may reduce N in code, but note it in the report.

Part A – Trends, Cost & Performance Indicators 25 marks

1. Explain bandwidth vs latency with one hardware example for each.
2. Manufacturing Cost: A 30 cm wafer costs \$2000 and yields 640 dies. If the die yield is 78.7% and packaging/test yield is 95%, compute the cost per packaged IC. Show steps and formulae.
3. Historical rule-of-thumb: “Bandwidth grows at least as the square of the improvement in latency.” Interpret and give a plausible reason (architectural or technological) for why this trend has held.

Part B – Availability & Dependability Scenario 25 marks

Your campus service runs on either (i) a single 12-core SMP server or (ii) a 3-node cluster with 4 cores each over Gigabit Ethernet. Discuss how each option behaves if one core/node fails, and outline one strategy per option to maintain service quality.

Part C – Amdahl's Law 25 marks

Use Amdahl's Law

4. For $\text{parallel} = 0.90$ and $\text{cores} \in \{1, 2, 4, 8\}$, compute speedup $S()$ and efficiency $E()$. Present as a table. Briefly comment on the diminishing returns.

5. Given parallel region = 0.92, what is the minimum core-count p needed to achieve $S() \geq 6$? Show algebraic steps.
6. Two teams optimize different parts: Team A increases parallel from 0.90→0.93 at $p=8$; Team B adds cores 8→16 at parallel=0.90. Which yields higher speedup? Justify numerically.

Part D – OpenMP Programming 25 marks

Complete the provided starter (code/omp_assignment.c). Keep $N=10,000,000$ (10 million) integers by default; reduce if memory-constrained and report the change.

1) Parallel Sum with Reduction

- Initialize array $A[i] = i \% 100$ (or random).
- Compute a sequential sum (baseline).
- Compute a parallel sum using `#pragma omp parallel for reduction(+:sum)`.
- Measure wall time with `omp_get_wtime()`.
- Run with threads $T \in \{1, 2, 4, 8\}$ (or up to your CPU's max). Record time, speedup, efficiency in results/measurements.csv.

2) Scheduling Experiment

- Create a synthetic workload loop with varying work per iteration (e.g., inner loop dependent on i).
- Compare schedules: static, dynamic, guided (keep chunk sizes reasonable).
- Report which schedule performs best and why (load balance vs overhead).

Academic Integrity

Write your own answers and code. You may discuss concepts, but sharing code or copying is not allowed. Cite any external sources briefly in your report.